BUSINESS PROBLEM

In [7]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

LOAD DATASET

In [8]:
```
df = pd.read_csv(r"D:\student_info.csv")
df
```

Out[8]:

	study_hours	student_marks
0	6.83	78.50
1	6.56	76.74
2	NaN	78.68
3	5.67	71.82
4	8.67	84.19
...	...	...
195	7.53	81.67
196	8.56	84.68
197	8.94	86.75
198	6.60	78.05
199	8.35	83.50

200 rows × 2 columns

In [9]:
```
df.head()
```

Out[9]:

	study_hours	student_marks
0	6.83	78.50
1	6.56	76.74
2	NaN	78.68
3	5.67	71.82
4	8.67	84.19

In [10]:
```
df.tail()
```

Out[10]:

	study_hours	student_marks
195	7.53	81.67
196	8.56	84.68
197	8.94	86.75
198	6.60	78.05
199	8.35	83.50

In [11]:
```
df.shape
```

Out[11]:
```
(200, 2)
```

DISCOVER AND VISUALIZE THE DATA TO GAIN INSIGHTS

In [12]:
```
df.info()
```

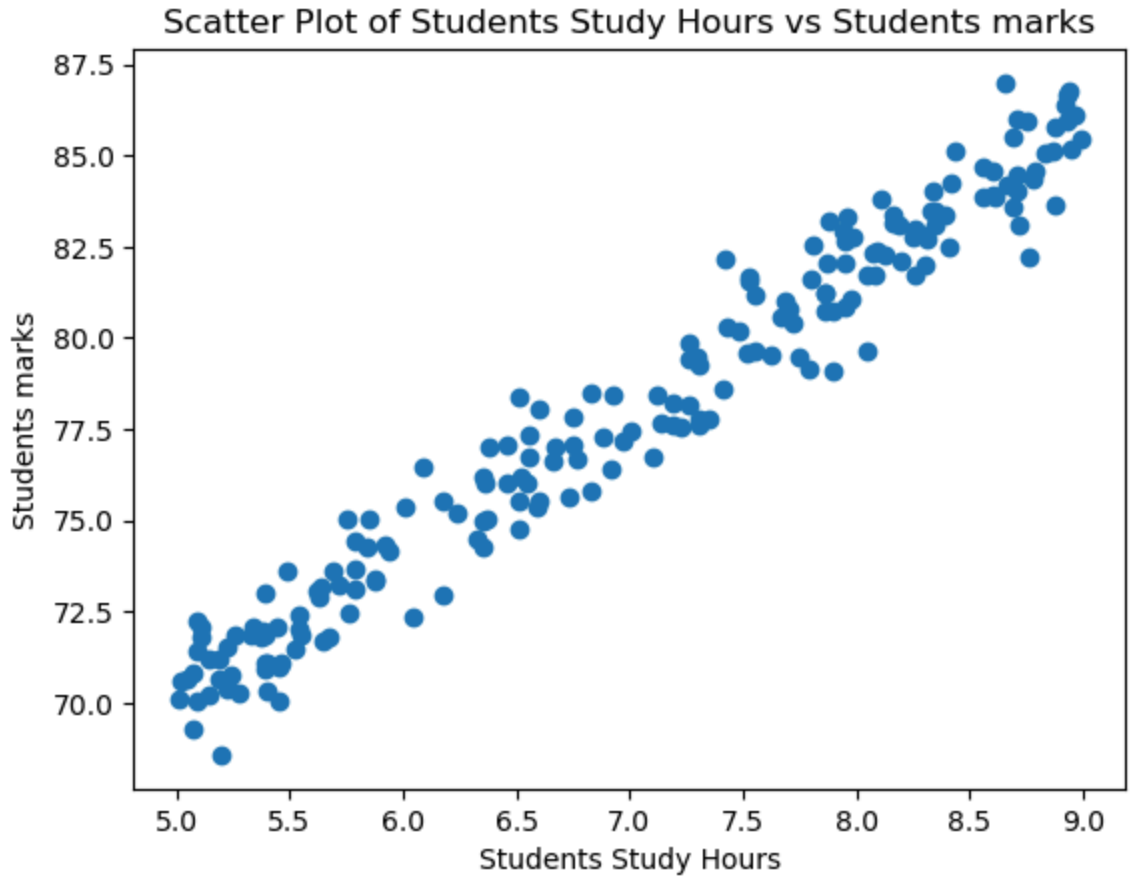
```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 200 entries, 0 to 199
Data columns (total 2 columns):
 #   Column         Non-Null Count  Dtype
---  ------         --------------  -----
 0   study_hours    195 non-null    float64
 1   student_marks  200 non-null    float64
dtypes: float64(2)
memory usage: 3.3 KB
```

In [13]:
```
df.describe()
```

Out[13]:

	study_hours	student_marks
count	195.000000	200.00000
mean	6.995949	77.93375
std	1.253060	4.92570
min	5.010000	68.57000
25%	5.775000	73.38500
50%	7.120000	77.71000
75%	8.085000	82.32000
max	8.990000	86.99000

In [14]:
```
plt.scatter(x = df.study_hours, y= df.student_marks)
plt.xlabel ("Students Study Hours")
plt.ylabel ("Students marks")
plt.title ("Scatter Plot of Students Study Hours vs Students marks")
plt.show()
```



DATA CLEANING

In [15]:
```
df
```

Out[15]:

	study_hours	student_marks
0	6.83	78.50
1	6.56	76.74
2	NaN	78.68
3	5.67	71.82
4	8.67	84.19
...	...	...
195	7.53	81.67
196	8.56	84.68
197	8.94	86.75
198	6.60	78.05
199	8.35	83.50

200 rows × 2 columns

In [16]:
```
df.isnull().sum()
```

Out[16]:
```
study_hours      5
student_marks    0
dtype: int64
```

In [17]:
```
df.mean()
```

Out[17]:
```
study_hours       6.995949
student_marks    77.933750
dtype: float64
```

In [18]:
```
df2 = df.fillna(df.mean())
```

In [19]:
```
df2.isnull().sum()
```

Out[19]:
```
study_hours      0
student_marks    0
dtype: int64
```

In [20]:
```
df2
```

Out[20]:

	study_hours	student_marks
0	6.830000	78.50
1	6.560000	76.74
2	6.995949	78.68
3	5.670000	71.82
4	8.670000	84.19
...	...	...
195	7.530000	81.67
196	8.560000	84.68
197	8.940000	86.75
198	6.600000	78.05
199	8.350000	83.50

200 rows × 2 columns

In [21]:
```
df2.head()
```

Out[21]:

	study_hours	student_marks
0	6.830000	78.50
1	6.560000	76.74
2	6.995949	78.68
3	5.670000	71.82
4	8.670000	84.19

In [22]:
```
df.head()
```

Out[22]:

	study_hours	student_marks
0	6.83	78.50
1	6.56	76.74
2	NaN	78.68
3	5.67	71.82
4	8.67	84.19

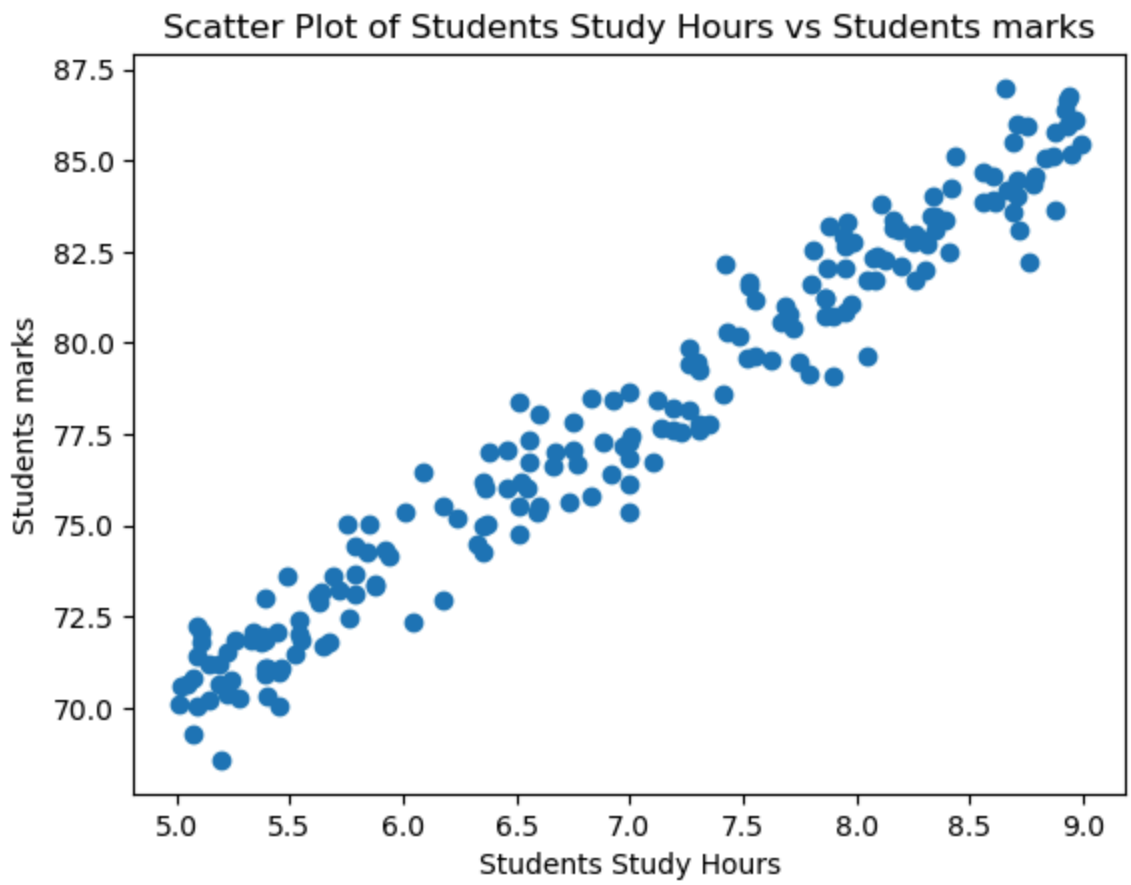
In [23]:
```
df2.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 200 entries, 0 to 199
Data columns (total 2 columns):
 #   Column         Non-Null Count  Dtype
---  ------         --------------  -----
 0   study_hours    200 non-null    float64
 1   student_marks  200 non-null    float64
dtypes: float64(2)
memory usage: 3.3 KB
```

In [24]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 200 entries, 0 to 199
Data columns (total 2 columns):
 #   Column         Non-Null Count  Dtype
---  ------         --------------  -----
 0   study_hours    195 non-null    float64
 1   student_marks  200 non-null    float64
dtypes: float64(2)
memory usage: 3.3 KB
```

In [25]:
```
plt.scatter(x = df2.study_hours, y= df2.student_marks)
plt.xlabel ("Students Study Hours")
plt.ylabel ("Students marks")
plt.title ("Scatter Plot of Students Study Hours vs Students marks")
plt.show()
```



SPLIT DATASET

In [26]:
```
x = df2.drop("student_marks", axis= "columns")
y = df2.drop("study_hours", axis = "columns")
print("shape of x = ", x.shape)
print("shape of y =", y.shape)
```

```
shape of x =  (200, 1)
shape of y = (200, 1)
```

In [27]:
```
x # x is independent variable
```

Out[27]:

	study_hours
0	6.830000
1	6.560000
2	6.995949
3	5.670000
4	8.670000
...	...
195	7.530000
196	8.560000
197	8.940000
198	6.600000
199	8.350000

200 rows × 1 columns

In [28]:
```
y # y is dependent variable
```

Out[28]:

	student_marks
0	78.50
1	76.74
2	78.68
3	71.82
4	84.19
...	...
195	81.67
196	84.68
197	86.75
198	78.05
199	83.50

200 rows × 1 columns

In [30]:
```
from sklearn.model_selection import train_test_split
x_train, x_test, y_train, y_test = train_test_split(x,y,test_size = 0.2, random_sta

print("Shape of x_train", x_train.shape)
print("Shape of y_train", y_train.shape)
print("Shape of x_test", x_test.shape)
print("Shape of y_test", y_test.shape)
```

```
Shape of x_train (160, 1)
Shape of y_train (160, 1)
Shape of x_test (40, 1)
Shape of y_test (40, 1)
```

In [31]:
```
x_train
```

Out[31]:

	study_hours
134	6.51
66	7.86
26	6.51
113	7.95
168	7.95
...	...
67	8.26
192	8.71
117	8.83
47	5.01
172	7.35

160 rows × 1 columns

In [32]:
```
x_train.shape
```

Out[32]:
```
(160, 1)
```

In [34]:
```
x_test.shape
```

Out[34]:
```
(40, 1)
```

In [35]:
```
y_train
```

Out[35]:

	student_marks
134	78.39
66	81.25
26	74.75
113	80.86
168	82.68
...	...
67	81.70
192	84.03
117	85.04
47	70.11
172	77.78

160 rows × 1 columns

In [36]:
```
y_train.shape
```

Out[36]: (160, 1)

In [37]:
```
y_test.shape
```

Out[37]: (40, 1)

SELECT A MODEL AND TRAIN ITT

In [41]:
```
# y=m*x+c

from sklearn.linear_model import LinearRegression
lr = LinearRegression()
```

In [42]:
```
lr
```

Out[42]:
```
▾ LinearRegression
▶ Parameters
```

In [43]:
```
lr.fit(x_train, y_train)
```

Out[43]:
```
▾ LinearRegression
▶ Parameters
```

In [44]:
```
lr.coef_ # m
```

Out[44]:
```
array([[3.93037294]])
```

In [45]:
```
lr.intercept_ # c
```

Out[45]:
```
array([50.45063632])
```

In [46]:
```
m = 3.93
c = 50.45
y = m * 10 + c
y
```

Out[46]:
```
89.75
```

In [47]:
```
m = 3.93
c = 50.45
y = m * 11 + c
y
```

Out[47]:
```
93.68
```

In [48]:
```
lr.get_params()
```

Out[48]:
```
{'copy_X': True,
 'fit_intercept': True,
 'n_jobs': None,
 'positive': False,
 'tol': 1e-06}
```

In [49]:
```
m = 3.93
c = 50.45
y = m * 13 + c
y
```

Out[49]:
```
101.54
```

In [50]:
```
lr.predict([[11]]).round(2)
```

```
D:\Users\3star\Lib\site-packages\sklearn\utils\validation.py:2749: UserWarning: X do
es not have valid feature names, but LinearRegression was fitted with feature names
  warnings.warn(
```

Out[50]:
```
array([[93.68]])
```

In [51]:
```
#lr.predict([[5]])[0][0].round(2)
```

In [52]:
```
y_pred = lr.predict(x_test)
y_pred
```

Out[52]:
```
array([[83.50507271],
       [70.84927186],
       [72.93236952],
       [85.35234799],
       [73.20749562],
       [84.48766595],
       [80.12495199],
       [81.85431608],
       [80.91102657],
       [82.20804964],
       [78.98514384],
       [84.84139951],
       [77.84533568],
       [77.68812077],
       [83.22994661],
       [85.78468901],
       [84.9593107 ],
       [72.61793968],
       [78.71001773],
       [79.18166248],
       [84.2911473 ],
       [85.6274741 ],
       [74.74034107],
       [81.3433676 ],
       [72.02838374],
       [80.40007809],
       [78.98514384],
       [82.09013845],
       [77.94732382],
       [82.24735337],
       [75.44780819],
       [84.60557713],
       [71.63534645],
       [75.48711192],
       [70.29901965],
       [78.98514384],
       [75.32989701],
       [84.52696967],
       [74.07217767],
       [71.4388278 ]])
```

In [56]:
```
pd.DataFrame(np.c_[x_test, y_test, y_pred], columns = ["study_hours", "student_mark
```

Out[56]:

	study_hours	student_marks_original	student_marks_predicted
0	8.410000	82.50	83.505073
1	5.190000	71.18	70.849272
2	5.720000	73.25	72.932370
3	8.880000	83.64	85.352348
4	5.790000	73.64	73.207496
5	8.660000	86.99	84.487666
6	7.550000	81.18	80.124952
7	7.990000	82.75	81.854316
8	7.750000	79.50	80.911027
9	8.080000	81.70	82.208050
10	7.260000	79.41	78.985144
11	8.750000	85.95	84.841400
12	6.970000	77.19	77.845336
13	6.930000	78.45	77.688121
14	8.340000	84.00	83.229947
15	8.990000	85.46	85.784689
16	8.780000	84.35	84.959311
17	5.640000	73.19	72.617940
18	7.190000	78.21	78.710018
19	7.310000	77.59	79.181662
20	8.610000	83.87	84.291147
21	8.950000	85.15	85.627474
22	6.180000	72.96	74.740341
23	7.860000	80.72	81.343368
24	5.490000	73.61	72.028384
25	7.620000	79.53	80.400078
26	7.260000	78.17	78.985144
27	8.050000	79.63	82.090138
28	6.995949	76.83	77.947324
29	8.090000	82.38	82.247353

	study_hours	student_marks_original	student_marks_predicted
30	6.360000	76.04	75.447808
31	8.690000	85.48	84.605577
32	5.390000	71.87	71.635346
33	6.370000	75.04	75.487112
34	5.050000	70.67	70.299020
35	7.260000	79.87	78.985144
36	6.330000	74.49	75.329897
37	8.670000	84.19	84.526970
38	6.010000	75.36	74.072178
39	5.340000	72.10	71.438828

FINE-TUNE YOUR MODEL

In [57]:
```
lr
```

Out[57]:
```
▾ LinearRegression
▶ Parameters
```

In [58]:
```
lr.score(x_test, y_test) # variance
```

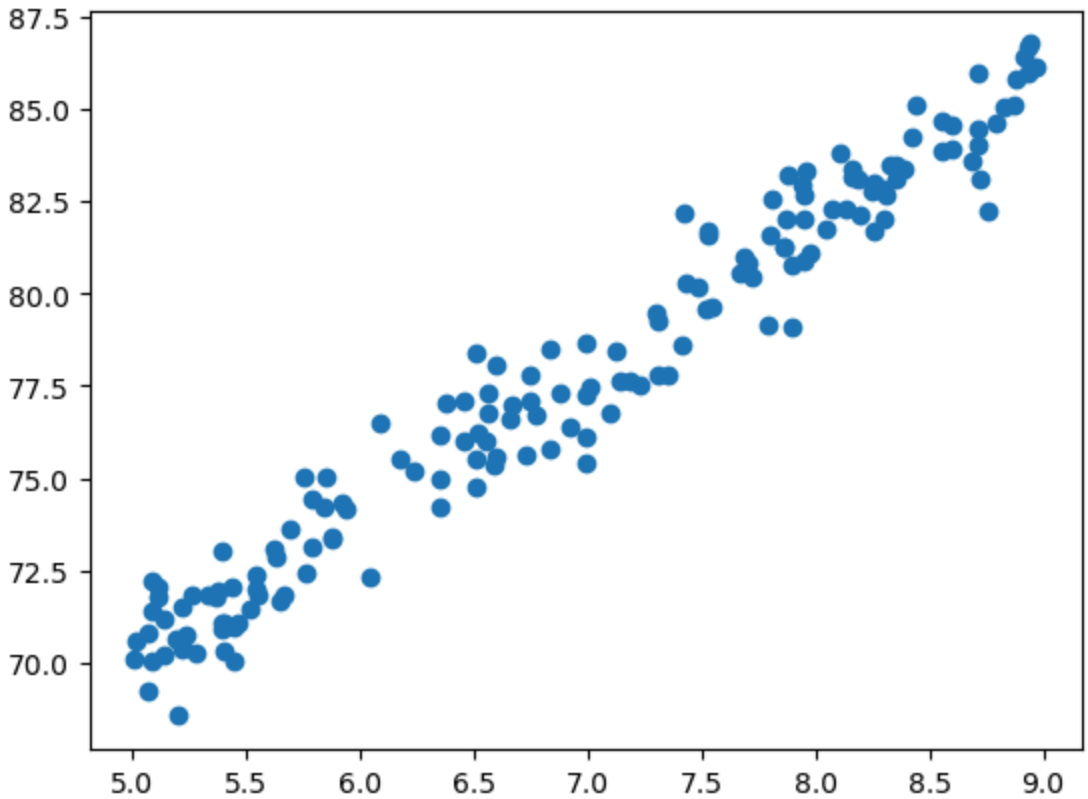
Out[58]:
```
0.9521841793508594
```

In [59]:
```
lr.score(x_train, y_train) # bias
```

Out[59]:
```
0.9584528455152638
```

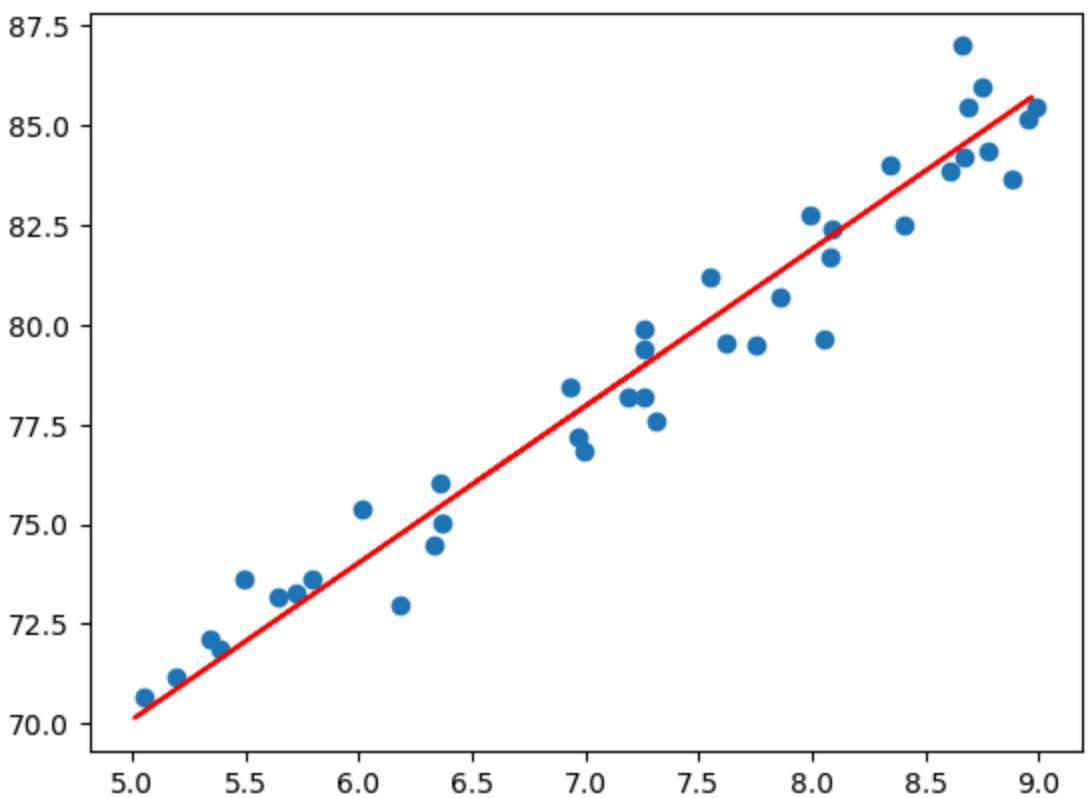
In [60]:
```
plt.scatter(x_train, y_train)
```

Out[60]:
```
<matplotlib.collections.PathCollection at 0x1f2be9eafd0>
```



In [61]:
```
plt.scatter(x_test, y_test)
plt.plot(x_train, lr.predict(x_train), color = "r")
```

Out[61]: [<matplotlib.lines.Line2D at 0x1f2be9d74d0>]



SAVE THE ML MODEL

In [64]:
```
import joblib   #create a pipeline
joblib.dump(lr, "SMP.pkl")
```

Out[64]:
```
['SMP.pkl']
```

In [65]:
```
model = joblib.load("SMP.pkl")
model
```

Out[65]:
```
▾ LinearRegression
▶ Parameters
```

In [ ]:

In [ ]:

In [ ]:

In [ ]: